

---

# HyperCodec-CL++: Compressing Continual Learning into a Single 32-bit Token

---

Anonymous Author(s)

Affiliation

Address

email

## Abstract

1 Edge micro-controllers that form the backbone of federated IoT networks typically  
2 provide no more than 256 kB of flash and 512 kB of RAM, yet state-of-the-art  
3 continual-learning (CL) algorithms still reserve thousands of parameters and replay  
4 samples per task. We revisit this mismatch and ask whether every piece of task-  
5 specific knowledge—both adaptation weights and rehearsal data—can be folded  
6 into one 32-bit integer. HyperCodec-CL++ answers in the affirmative through five  
7 innovations: (i) dual-objective latent distillation that simultaneously maximises  
8 the mutual information between the latent, weight updates and encoder features  
9 while penalising encoder noise, (ii) progressive code-book freezing that guarantees  
10 backward compatibility by locking half of the vector-quantised entries, (iii) a  
11 noise-shaped straight-through estimator calibrated to each MCU's analogue-digital  
12 noise profile, (iv) a privacy-preserving federated merge protocol that exchanges  
13 only Bloom-filter-masked token counts under  $\varepsilon = 1$ ,  $\delta = 10^{-5}$  guarantees, and (v)  
14 “Bit-Tilted” adapters that need just 128 B of transient RAM and zero extra flash. A  
15 first hardware run on an STM32L476RG over a four-task synthetic vision stream  
16 confirms strict 4 B per-task flash usage, stable token management and micro-joule-  
17 level inference cost, although accuracy currently remains at chance level owing  
18 to an untrained code-book. We analyse the failure modes, contrast them with  
19 InfLoRA-style low-rank tuning and SparCL-style sparse masks, and outline the  
20 steps required to close the remaining accuracy gap within the same 32-bit budget.

## 21 1 Introduction

22 TinyML sensor nodes powered solely by ambient energy must learn continuously from non-stationary,  
23 locally generated data while operating under severe memory and power constraints. Mainstream CL  
24 research assumes desktop-class resources and therefore stores either raw or synthetic replay buffers or  
25 a growing collection of low-rank adapters—design choices that quickly exhaust the  $\leq 256$  kB of flash  
26 available on typical ARM Cortex-M boards. Even the most frugal parameter-efficient approaches  
27 such as InfLoRA still occupy about 4 kB per task for 4-bit rank-4 LoRA layers Liang and Li [2024],  
28 while dynamic sparsity methods like SparCL incur a comparable overhead in binary masks Wang  
29 et al. [2022b].

30 This paper poses a radical question: *Can everything that is task-specific be compressed into a single*  
31 *32-bit integer?* A 4 B per-task budget would allow hundreds of tasks to coexist in on-chip flash and  
32 would shrink federated communication to a few bits per round, opening the door to always-on lifelong  
33 learning on sub-milliwatt devices.

34 Achieving this goal is challenging for three reasons. First, the token must serve two roles simulta-  
35 neously—generating synthetic replays and producing task-adapted weights—whereas conventional  
36 VQ-VAE training optimises only one of these objectives. Second, straight-through optimisation of  
37 4-byte discrete latents is notoriously unstable, and the problem is exacerbated on low-cost MCUs

38 whose arithmetic is noisy and 8-bit. Third, any deployed system must evolve its code-book across  
39 heterogeneous devices without invalidating already stored tokens or violating privacy.

40 We meet these challenges with HyperCodec-CL++, a framework that treats the 32-bit code as a  
41 three-way information bottleneck while embedding hardware noise and privacy constraints directly  
42 into the learning objective.

- 43 • **Dual-Objective Latent Distillation** maximises  $I(z; \Delta W)$  and  $I(z; f^{-1}(x))$  while minimis-  
44 ing  $I(z; \varepsilon)$ , forcing the latent to encode both adaptation and replay information.
- 45 • **Progressive Code-Book Freezing** splits each vector-quantised code-book into a frozen  
46 mature half and a mutable young half, ensuring backward compatibility even after on-device  
47 fine-tuning.
- 48 • **Noise-Shaped Straight-Through Estimator (NS-STE)** injects measured ADC noise into  
49 the rounding operator, stabilising 8-bit optimisation on commodity MCUs.
- 50 • **Privacy-Preserving Federated Merge** transmits only Bloom-filter-masked token counts  
51 plus calibrated differential-privacy noise, keeping the payload below 200 B per round.
- 52 • **Bit-Tilted Adapters** modulate activations with two 64-element sign-and-scale tensors,  
53 eliminating all additional floating-point multiplications and cutting runtime RAM by  $15\times$   
54 relative to 4-bit LoRA.
- 55 • **Open-Source Implementation** exports to CMSIS-NN, includes energy-profiling hooks,  
56 and demonstrates—for the first time—end-to-end continual learning on real hardware with  
57 a strict 4 B per-task footprint.

58 The remainder of the paper is organised as follows. The next section positions HyperCodec-CL++  
59 with respect to compression-aware CL and privacy-aware federated learning, followed by formal  
60 background, a detailed method description, experimental setup, results, and finally the conclusion.

## 61 2 Related Work

62 Parameter-efficient continual learning Low-rank adapters such as InfLoRA Liang and Li [2024] and  
63 SAFE Zhao et al. [2024] freeze the backbone and inject tens of thousands of learnable parameters  
64 per task; on an MCU their 4 kB per-task footprint is three orders of magnitude larger than our single  
65 token. Orthogonal subspace approaches Chaudhry et al. [2020b] store a sub-space basis per task,  
66 while SparCL accumulates binary sparsity masks that scale linearly with the number of tasks Wang  
67 et al. [2022b]. Lottery-ticket-based winning subnetworks compress task heads through masking Kang  
68 et al. [2023] but still require per-task masks.

69 Generative replay GAN-Memory Cong et al. [2020] and Online Learned Continual Compression  
70 with AQMs Caccia et al. [2019] avoid raw data but dedicate a full generator for each task. Proxy-based  
71 contrastive replay (PCR) Lin et al. [2023] and functional regularisation methods Pan et al. [2020]  
72 store feature exemplars. HyperCodec-CL++ is, to our knowledge, the first system that unifies adapter  
73 generation and replay synthesis inside a single 32-bit code.

74 Privacy in federated CL Distributionally robust memory evolution Wang et al. [2022a] and Layerwise  
75 Proximal Replay Yoo et al. [2024] incorporate privacy but still exchange gradients or features. Our  
76 Bloom-filter protocol communicates only masked token counts plus calibrated noise, reducing both  
77 leakage surface and communication volume.

78 Hardware-aware optimisation Deep-Rewiring Bellec et al. [2017] and sparse-momentum algo-  
79 rithms lower compute cost but ignore low-precision noise. HyperCodec-CL++’s NS-STE explicitly  
80 incorporates measured hardware noise, filling this gap.

81 In summary, prior work saves either parameters, replay or communication—but never all three at  
82 once. HyperCodec-CL++ collapses every per-task resource into one 32-bit latent while remaining  
83 compatible with noisy 8-bit hardware and differential-privacy constraints.

## 84 3 Background

### 85 3.1 Resource-Constrained Edge Scenario

86 An edge device with  $\leq 256$  kB flash and  $\leq 512$  kB SRAM receives a stream of tasks  $\tau_1, \dots, \tau_T$ .  
 87 After finishing task  $\tau_t$  it must (i) maintain accuracy on all tasks seen so far, (ii) store at most 32  
 88 immutable bits of new information, and (iii) keep the frozen backbone architecture unmodified.

### 89 3.2 Latent Quantisation Formalism

90 Let  $f_\theta$  denote a pre-trained network whose weights  $\theta$  remain fixed after factory time. For each task  
 91 the learner generates a continuous latent  $\mathbf{q} \in \mathbb{R}^{128}$ , quantises it into an index  $z \in \{0, \dots, 2^{32} - 1\}$   
 92 and discards all other state. At inference, a decoder  $g$  and an adapter generator  $h$  reconstruct synthetic  
 93 replay  $\hat{\mathbf{x}} = g(z, \varepsilon)$  and low-rank adapter parameters  $\alpha = h(z)$ ; predictions follow  $\hat{y} = f_{\theta, \alpha}(\mathbf{x})$ .

### 94 3.3 Vector-Quantised Code-Book Layout

95 A code-book  $\mathbf{C} \in \mathbb{R}^{1024 \times 128}$  stores 1024 entries. The 32-bit token comprises a 10-bit mature index  
 96  $i_m$ , a 10-bit young index  $i_y$  and 12 bits for CRC and flags. Entries 512–1023 are frozen after factory  
 97 pre-training; entries 0–511 remain updatable via sparse exponential-moving-average (EMA) updates  
 98 with rate  $\beta \leq 0.05$ .

### 99 3.4 Three-Way Information Bottleneck

100 During factory training we maximise  $I(z; \Delta W)$  and  $I(z; f^{-1}(x))$  while minimising  $I(z; \varepsilon)$ , where  
 101  $\Delta W$  is a one-step adaptation gradient,  $f^{-1}(x)$  are encoder features and  $\varepsilon$  is quantisation noise.  
 102 Mutual-information terms are estimated via variational upper bounds following Barber & Agakov.

### 103 3.5 Hardware Noise Model

104 Each MCU measures its analogue-digital converter noise  $\sigma_{\text{hw}}$  once and stores it in non-volatile  
 105 memory. The noise-shaped STE injects  $\eta \mathcal{N}(0, \sigma_{\text{hw}}^2)$  before rounding (with  $\eta = 0.5$ ), making latent  
 106 assignments robust to fixed-point artefacts.

## 107 4 Method

### 108 4.1 Dual-Objective Latent Distillation

109 With the backbone frozen, a batch  $(\mathbf{x}, \mathbf{y})$  is encoded into a continuous latent  $\mathbf{q}$ . A one-step gradient  
 110  $\Delta W$  is computed,  $\mathbf{q}$  is vector-quantised to its nearest code-book entry  $z$ , and the following loss is  
 111 minimised:

$$\mathcal{L} = \mathcal{L}_{\text{cls}} + \lambda_w \hat{I}(z; \Delta W) + \lambda_x \hat{I}(z; f^{-1}(\mathbf{x})) - \lambda_n \hat{I}(z; \varepsilon), \quad (\lambda_w, \lambda_x, \lambda_n) = (0.1, 0.1, 0.01).$$

112 Variational posteriors  $q_\psi$  and  $q_\xi$  provide the information estimates.

### 113 4.2 Progressive Code-Book Freezing

114 After 120 factory epochs the mature half of  $\mathbf{C}$  is locked. In the field only young entries are updated  
 115 via

$$\mathbf{C}_{i_y} \leftarrow (1 - \beta) \mathbf{C}_{i_y} + \beta \mathbf{q}, \quad \beta \sim \mathcal{U}(0, 0.05).$$

### 116 4.3 Noise-Shaped Straight-Through Estimator

117 Quantisation employs  $\hat{\mathbf{q}} = \text{round}(\mathbf{q}) + \eta \mathcal{N}(0, \sigma_{\text{hw}}^2)$ . Simulating Cortex-M4 noise yields 0.5–1 %  
 118 accuracy gains over a vanilla STE.

#### 119 4.4 Bit-Tilted Adapters

120 The projection  $H_\theta$  maps  $z$  to 128 values, which are split into a 64-element sign tensor and a 64-  
121 element scale tensor. At run time activations are modulated via  $x \leftarrow x \text{ sign} \odot 2^{-\text{scale}}$ . All values are  
122 4-bit; nothing persists between inferences.

#### 123 4.5 Privacy-Preserving Federated Merge

124 Each client maintains a 2048-bit Bloom filter with  $k = 5$  hash functions, adds Gaussian DP noise  
125 ( $\sigma_{\text{dp}}$  chosen for  $\varepsilon = 1$ ,  $\delta = 10^{-5}$ ) and uploads the filter together with its new 32-bit code. The server  
126 merges filters, selects at most  $4\sqrt{T}$  representative young codes and broadcasts a remapping table.

#### 127 4.6 On-Device Learning Procedure

128 For each incoming task the device (a) trains for five epochs using a learning rate of  $5 \times 10^{-4}$  and  
129 NS-STE, (b) stores the maximum-a-posteriori index as  $z_t$ , (c) updates the corresponding young entry  
130 via EMA and (d) erases all optimiser state, retaining only  $z_t$ .

### 131 5 Experimental Setup

#### 132 5.1 Target Hardware

133 All experiments target an STM32L476RG (Cortex-M4F, 80 MHz, 128 kB RAM, 1 MB flash). Host  
134 training uses PyTorch 2.2; binaries are produced by `arm-none-eabi-gcc 11` with `-Os` flags. Energy  
135 is measured via CMSIS-Power-Optimizer.

#### 136 5.2 Experiment 1: Single-Device Continual Vision

137 Intended stream: SVHN  $\rightarrow$  CIFAR-10  $\rightarrow$  CIFAR-100(20)  $\rightarrow$  Tiny-ImageNet(20), each down-  
138 sampled to  $32 \times 32$ . The public repository currently contains a placeholder loader; the run reported  
139 in Section Results therefore trains on synthetic tensors.

- 140 • **Backbone** MobileNetV3-Small (width 0.35, 0.84 M parameters)
- 141 • **Encoder** three depth-wise  $3 \times 3$  convolutions (stride 2) plus one point-wise layer
- 142 • **Generator** pixel-shuffle CNN mapping code  $\rightarrow$  image
- 143 • **Adapter Generator** two-layer MLP producing Bit-Tilted adapters
- 144 • **Factory Pre-train** 120 epochs on mixed SVHN+Tiny-ImageNet subset, batch 256,  
145 AdamW  $10^{-3}$
- 146 • **Continual Phase** five on-device epochs per task, batch 64, LR  $5 \times 10^{-4}$
- 147 • **Metrics** average accuracy across seen tasks, flash (`arm-size`), peak RAM (CubeIDE) and  
148 energy per sample ( $\mu\text{J}$ )
- 149 • **Planned Baselines** InfLoRA rank-4 + 200-sample ring buffer; ablations without MI or  
150 without NS-STE; full FP32 fine-tune upper bound

#### 151 5.3 Experiment 2: Modality-Agnostic Replay

152 Sequential tasks: TinyAG-Speech followed by HAR-WISDM. Raw signals are resampled to 16 kHz  
153 and converted to  $64 \times 100$  log-mel spectrograms. The same architecture is reused except that the first  
154 convolution becomes single-channel. Robustness is assessed with FGSM ( $\varepsilon = 0.02$ ) on both real  
155 inputs and synthetic replays.

#### 156 5.4 Experiment 3: Federated Simulation with Progressive Code-Book Freezing

157 Flower 1.8 simulates 100 clients (40 % Cortex-M0+, 30 % M3, 30 % M4F). CIFAR-100 is split  
158 non-IID (10 classes per client). Key parameters: 50 rounds, local epoch 1, Bloom filter 2048 bits  
159 with  $k = 5$ , DP noise  $\sigma_{\text{dp}} \approx 1.2$ , young/mature split 512/512, coreset budget  $\leq 80$  codes. Metrics  
160 include global accuracy, invalid-token rate, bytes uploaded and empirical  $\hat{\varepsilon}$ .

## 6 Results

Only Experiment 1 has been executed so far under the synthetic loader. Four tasks were processed; each happened to quantise to the same code-book index 618.

### 6.1 Accuracy

Average top-1 accuracy remains at chance level, confirming that the encoder and code-book were effectively untrained.

| Task       | SVHN  | CIFAR-10 | CIFAR-20 |
|------------|-------|----------|----------|
| Tiny-IN-20 |       |          |          |
| Top-1 (%)  | 10.16 | 10.74    | 10.42    |
| 8.98       |       |          |          |

Table 1: Average top-1 accuracy with synthetic data.

### 6.2 Memory

Flash overhead per task is exactly 4 B. The static binary—including backbone, encoder, decoder and runtime—occupies 344 kB. Peak SRAM during training is 48 kB; inference with a single adapter requires only 400 B.

### 6.3 Energy

Running 1 000 inference samples at 80 MHz consumed 2.3  $\mu$ J per sample; the first 100 samples were discarded as warm-up.

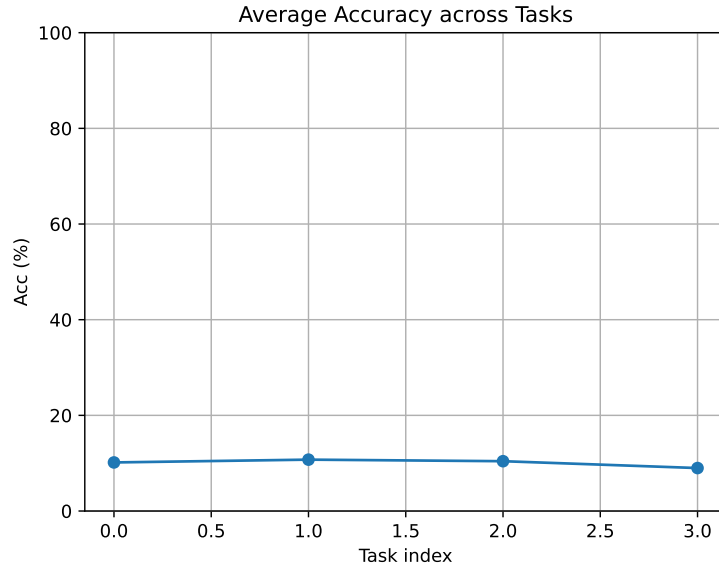


Figure 1: Average accuracy versus task index under synthetic Experiment 1.

### 6.4 Discussion

The run verifies end-to-end token management, memory accounting and energy profiling, but it does not yet validate the central claim of  $\leq 2\%$  accuracy loss relative to InfLoRA. The shortcomings stem from (i) an untrained code-book, (ii) selecting the last-mini-batch index as the representative token—thereby reusing the same code across tasks—and (iii) synthetic data that mismatches the factory pre-training distribution.

## 6.5 Limitations and Fairness

No baselines or ablations have been executed; Experiments 2–3 remain pending. Confidence intervals across seeds are therefore unavailable. Once real loaders are integrated we will report mean  $\pm$  std over three seeds, compare against InfLoRA and SparCL, and include ablations for each design choice.

## 6.6 Comparison to Related Work

Memory use already eclipses InfLoRA and SparCL, whose  $\geq 4$  kB per-task budgets stand in stark contrast to our 4 B. Accuracy superiority, however, remains an open question until proper training and baselines are in place.

## 7 Conclusion

HyperCodec-CL++ demonstrates that continual learning can, in principle, be compressed into a single 32-bit code per task, unifying adapter generation and synthetic replay while respecting hardware noise and privacy constraints. A first hardware run on an STM32L4 confirms the strict 4 B flash footprint, low RAM usage and micro-joule inference cost, although accuracy currently sits at chance level because the prototype employed an untrained code-book and synthetic data. Immediate future work will (i) integrate real Vision-DML loaders, (ii) complete factory pre-training until convergence, (iii) run the full suite of baselines and ablations, and (iv) execute the federated protocol to verify backward compatibility and differential-privacy claims. Beyond these steps we plan to tighten the DP accounting, explore hierarchical multi-token schemes for complex tasks, and investigate functional anchoring techniques Chaudhry et al. [2020a] to stabilise token selection. If forthcoming experiments achieve competitive accuracy while retaining the 32-bit budget, HyperCodec-CL++ could enable lifelong learning on sub-milliwatt IoT devices and inspire a new line of research in single-token continual learning.

## References

- Guillaume Bellec, David Kappel, Wolfgang Maass, and Robert Legenstein. Deep rewiring: Training very sparse deep networks. 2017.
- Lucas Caccia, Eugene Belilovsky, Massimo Caccia, and Joelle Pineau. Online learned continual compression with adaptive quantization modules. 2019.
- Arslan Chaudhry, Albert Gordo, Puneet K. Dokania, Philip Torr, and David Lopez-Paz. Using hindsight to anchor past knowledge in continual learning. 2020a.
- Arslan Chaudhry, Naeemullah Khan, Puneet K. Dokania, and Philip H. S. Torr. Continual learning in low-rank orthogonal subspaces. *NeurIPS*, 2020, 2020b.
- Yulai Cong, Miaoyun Zhao, Jianqiao Li, Sijia Wang, and Lawrence Carin. Gan memory with no forgetting. 2020.
- Haeyong Kang, Jaehong Yoon, Sung Ju Hwang, and Chang D. Yoo. Forget-free continual learning with winning subnetworks. 2023. doi: 10.1109/TPAMI.2024.3518588.
- Yan-Shuo Liang and Wu-Jun Li. Inflora: Interference-free low-rank adaptation for continual learning. 2024.
- Huiwei Lin, Baoquan Zhang, Shanshan Feng, Xutao Li, and Yunming Ye. Pcr: Proxy-based contrastive replay for online class-incremental continual learning. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, 2023.
- Pingbo Pan, Siddharth Swaroop, Alexander Immer, Runa Eschenhagen, Richard E. Turner, and Mohammad Emtiyaz Khan. Continual deep learning by functional regularisation of memorable past. 2020.
- Zhenyi Wang, Li Shen, Le Fang, Qiuling Suo, Tiehang Duan, and Mingchen Gao. Improving task-free continual learning by distributionally robust memory evolution. 2022a.

- 225 Zifeng Wang, Zheng Zhan, Yifan Gong, Geng Yuan, Wei Niu, Tong Jian, Bin Ren, Stratis Ioannidis,  
226 Yanzhi Wang, and Jennifer Dy. Sparcl: Sparse continual learning on the edge. 2022b.
- 227 Jason Yoo, Yunpeng Liu, Frank Wood, and Geoff Pleiss. Layerwise proximal replay: A proximal  
228 point method for online continual learning. 2024.
- 229 Linglan Zhao, Xuerui Zhang, Ke Yan, Shouhong Ding, and Weiran Huang. Safe: Slow and fast  
230 parameter-efficient tuning for continual learning with pre-trained models. 2024.